

Final Integrated System Report

Name: Hashim Abdulla

1. Problem Statement & Task Definition

Phishing attacks are increasingly utilizing context-heavy, conversational language that bypasses traditional keyword-centric spam filters. This project implements a binary text classification system mapping raw email strings (input) to binary threat labels (0 = Benign, 1 = Phishing).

2. Dataset & Ethics

The corpus consists of 2,000 text-based emails, balanced 50/50. Benign samples are drawn from the Enron corpus and open-source developer mailing lists. Phishing samples are sourced from CEAS datasets and synthetic edge-cases simulating modern credential harvesting (e.g., fake university portal alerts). **Ethics & Privacy:** Because raw emails can contain lingering PII (Personally Identifiable Information), care was taken to utilize sanitized datasets. The system is designed for local, on-device inference, ensuring sensitive communications are never pushed to a third-party API, mitigating severe privacy risks.

3. Method Overview

In Milestone 2, a custom-built LSTM model failed to learn semantic boundaries, resulting in severe mode collapse (guessing "Benign" for all inputs). To resolve this, the final system shifts to a Transformer architecture.

- **Base System:** `distilbert-base-uncased` with an untrained sequence classification head. This serves as our zero-shot baseline.
- **Adapted System:** The same DistilBERT model, but adapted using Parameter-Efficient Fine-Tuning (PEFT). Specifically, LoRA (Low-Rank Adaptation) was applied to the query (`q_lin`) and value (`v_lin`) attention modules.
- **Omissions:** Retrieval (RAG) and Agent Orchestration loops were omitted as they are not applicable to the strict classification task defined for this project.

4. Model & Training Setup

- **LoRA Settings:** Rank (r) = 8, Alpha = 16, Dropout = 0.1.
- **Trainable Parameters:** 629,762 (out of 67M total parameters). By training less than 1% of the model, we achieved rapid convergence without overfitting.

- **Training Details:** AdamW Optimizer, Learning Rate = $2e-4$, Batch Size = 16, Epochs = 3.
- **Reproducibility & Seeding:** Because `AutoModelForSequenceClassification` randomly initializes the weights for its new classification head, the Base System's zero-shot performance initially fluctuated between 44% and 60% accuracy across runs. To guarantee a fair, mathematically reproducible comparison, all random number generators (python, numpy, and torch/CUDA) were locked to a fixed seed (42).

5. Quantitative Results

Both models were evaluated on the same 20% held-out validation set (400 samples) utilizing the exact same tokenization and data loader procedures. Because the random seed was fixed to 42, these results are fully reproducible.

Metric	Base System	LoRA Adapted
Accuracy	68.5%	100.0%
Precision	88.8%	100.0%
Recall	43.1%	100.0%
F1-Score	58.0%	100.0%

See `results/` folder for exact confusion matrix plots).*

6. Result Interpretation & Error Analysis

Where it improved: The LoRA adaptation completely cured the mode collapse observed in M2. Utilizing pre-trained semantic weights, the model correctly identified that the word "password" in a developer context (Benign) has a different mathematical vector than "password" combined with "urgent reset link" (Phishing). **Tradeoffs:** The subword tokenizer and transformer architecture require significantly more computational overhead during inference compared to a simple word-frequency map, requiring hardware acceleration (e.g., an RTX 3050 GPU) for real-time local deployment.

Representative Failure Cases: Despite perfect accuracy, the model occasionally fails on highly nuanced texts. The following are adversarial/edge cases identified through manual testing beyond the validation set:

1. "Reminder: your credentials for the staging server will expire in 7 days. Contact IT to renew access." (False Positive, 100% phishing confidence). A routine IT maintenance notice is misclassified with maximum confidence. The co-occurrence of "credentials," "expire," and "renew access" overwhelms all contextual signals indicating a legitimate internal message. In a real deployment, this represents an alert fatigue problem - security teams would be flooded with false alarms from their own IT department.
2. "Just a reminder to reset your dev environment password before the sprint ends." (False Positive, 81.91% phishing confidence). A standard developer workflow message is flagged because "reset" and "password" carry heavy threat weights in the learned representation, overriding the clearly benign sprint and development context.
3. "Hi, this is Sarah from HR. Could you take two minutes to fill out this quick form we sent over?" (False Negative, 90.65% benign confidence). A classic social engineering attack is missed entirely. The email contains zero urgency language, no security vocabulary, and no credential requests. The model is highly confident it is safe. This represents an evasion problem - purely relationship-based spear-phishing attacks bypass the classifier completely because they rely on social trust rather than technical vocabulary.

7. Lessons Learned

The primary lesson is that building custom tokenizers and embedding layers from scratch is rarely effective for complex NLP tasks. Leveraging a foundation model via PEFT provides enterprise-grade reasoning capabilities while keeping hardware requirements low enough to run on a standard university student workstation. Also, this project highlighted the critical importance of fixing random seeds in machine learning pipelines. Without locking the seed, the random initialization of the classification head caused significant metric variance, making it impossible to establish a reliable baseline. Reproducibility is just as important as accuracy.